

Take-Home Assignment — Shipment Events Service

Senior / Staff Backend Engineer · TransVirtual · Candidate brief

Overview

Build a service that ingests shipment tracking events from carriers — including duplicates and out-of-order delivery — and serves an accurate shipment timeline. We care most that it runs end-to-end as a workable demo, exercised by replay tooling you build yourself over the event data we provide.

What we provide

- `carrier_events.csv` — about 1,000 carrier events with columns: `event_id`, `shipment_reference`, `status`, `event_timestamp`. Duplicates and out-of-order batches are deliberately baked in.

That is all. The webhook payload format, the replay tooling, the environment, and every other choice are yours — and part of what we evaluate.

Requirements

1. Ingestion (Python/FastAPI): a webhook endpoint accepting carrier events (design your own JSON payload carrying the CSV fields). Ingestion must be idempotent by `event_id`, and out-of-order events must not corrupt the derived shipment state.
2. Data model (PostgreSQL): shipments and their events; the current status must be derivable correctly regardless of arrival order. Document your indexing rationale (in the schema or the ADR).
3. Query API: a shipment timeline endpoint (ordered event history plus current status) and a shipment list filterable by status.
4. Replay tooling: build a script that fires the `carrier_events.csv` events at your webhook — concurrency encouraged. After a full replay, duplicates and out-of-order batches included, every shipment's timeline and current status must be correct.
5. ADR (about a page): the key trade-offs you made, plus a short written design note on how you would move processing to an async pipeline (design only, no implementation).
6. Tests: at minimum, idempotency and out-of-order handling.
7. Workable demo: the service and database start with clearly documented commands (a single command is ideal); run your replay script; inspect results via the OpenAPI docs (`/docs`) or provided curl examples.

Ground rules

- Timebox: 4–6 hours of effort, due within 2 days of receiving this brief (a 3rd day is available on request). Stop at the timebox and write down what you would do next — unfinished-but-well-reasoned beats gold-plated.
- AI tools (Claude Code, Copilot, Cursor, etc.) are allowed and encouraged — we work AI-first. Include a short AI-usage note: which tools, what was generated vs. hand-written, and at least one thing the AI got wrong and how you caught it.
- Tooling is your call — languages are fixed by the requirements, but frameworks, libraries, project layout, and how you run things locally are up to you. Choose deliberately; we will ask why. Skip authentication, deployment, and anything not listed in the requirements.
- You will demo this live (about 5 minutes) at the start of your final interview and extend it during the session — build something you can talk about.

What to submit

- A Git repository link (or zip) with the full source.
- `README.md`: how to run the demo (the exact commands), key decisions — including your tooling choices and why — and your “what I’d do next” list.
- Your AI-usage note (in the `README` or separate).
- ADR as described above.

How we evaluate

- The demo is the deliverable: if we cannot run your project with the documented commands, we will ask you once to fix the setup; if it still doesn't run, we can't progress the application. The demo matters more than any individual feature.
- We look at: correctness after a full replay · data modeling and idempotency design · setup and tooling choices · code clarity · API design · how well the ADR communicates trade-offs · judgment in how you used AI tools.
- In the final interview you'll demo the service, defend the design as we scale the constraints (10× volume, multi-tenancy, partial failures, legacy migration), and discuss how you'd grow a young team around these standards.